Follow-up Assignment:

The Use of Type Annotations in Data Validation and JSON Serialization

November 2, 2026

1. In the `validation` package, declare a public annotation type by creating the interface `NotBlank` using the `@interface` construct.
   - Specify the annotation's lifetime so it is available during program execution, ensuring it can be read via reflection by setting the retention to `RetentionPolicy.RUNTIME`.
   - Limit where the annotation may be used to class fields only by indicating `ElementType.FIELD` as the sole allowed element type.
2. In the `validation` package, declare a public annotation type by creating the interface `NotNull` using the `@interface` construct.
   - Specify the annotation's lifetime so it is available during program execution and can be read via reflection by setting the retention to `RetentionPolicy.RUNTIME`.
   - Limit where the annotation may be used to class fields only by indicating `ElementType.FIELD` as the sole allowed element type.
3. In the `validation` package, declare a public annotation type by creating the interface `Range` using the `@interface` construct.
   - Specify the annotation's lifetime so it is available during program execution and can be read via reflection by setting the retention to `RetentionPolicy.RUNTIME`.
   - Limit where the annotation may be used to class fields only by indicating `ElementType.FIELD` as the sole allowed element type.
   - Define two annotation elements of type `int` named `min` and `max`, representing the minimum and maximum admissible value.
   - Set default values: `min` as `Integer.MIN_VALUE` and `max` as `Integer.MAX_VALUE`, allowing flexible usage without requiring these parameters every time.
4. In the `model` package, declare a public class `User` that represents a system user. Plan fields and accessors so that the class can be inspected via reflection and validated using annotations from the `validation` package.
   - Add a private identifier field `id` of type `long`, mark it as `final`, and impose a minimum value constraint. Use `@Range(min = 1)` to require a positive identifier.

- Add a private username field `username` of type `String` and require a non-blank value. Use `@NotBlank` to prevent empty or whitespace-only values.
- Add a private email field `email` of type `String` and require a non-null reference. Use `@NotNull` to enforce presence (without format verification).
- Add a private age field `age` of type `int` and restrict the admissible range of values. Use `@Range(min = 18, max = 100)` to allow only ages within the given interval.
- Add a private field `password` of type `String` storing the user password without validation annotations. Leave this field unannotated so it is not covered by annotation-based validation rules.
- Implement a public constructor initializing all fields: `id`, `username`, `email`, `age`, and `password`. Assign the passed arguments to the corresponding fields, ensuring a consistent object state from creation.
- Provide public accessor methods (getters) for each field to read the object state by adding:
  - `getId()`,
  - `getUsername()`,
  - `getEmail()`,
  - `getAge()`,
  - `getPassword()`.

5. Present your results to the instructor.

6. In the `processing` package, define a checked exception `ValidationException` for signaling validation errors by creating a class `ValidationException` extending `Exception` and add a constructor accepting an error message; use it in all cases of rule violations.

7. In the `processing` package, declare a public class `Validator` with a public static method `validate(Object obj)` that is declared to throw `ValidationException` and `IllegalAccessException`. The method is to serve as an entry point for validating any object based on annotations placed on its fields.

   - In `validate` method, obtain all declared fields of the object's class and iterate over them, ensuring access to private fields. Call `getDeclaredFields()` on the class obtained from `obj`. For each field, call `setAccessible(true)` to enable reading private fields.
   - Implement handling of the `@NotNull` annotation by detecting its presence on a field and verifying that the value is not `null`.
   - Implement handling of the `@NotBlank` annotation for textual fields, rejecting `null`, empty strings, and strings consisting only of whitespace.
   - Implement handling of the `@Range` annotation by verifying that a numeric field's value lies within the specified interval.

8. In the `processing` package, declare a public class `ObjectProcessor`, responsible for processing objects through validation and serialization. The class is to be the main integration point for the validation and serialization modules.

   - Implement a public method `process(Object obj)` returning a `String`, which will perform validation within a controlled `try-catch` block. Use the argument `obj` as the input to the processing pipeline, enabling handling of any object type. The method should:
     - print a message to the console indicating the start of validation;
     - call `Validator.validate(obj)`;
     - after a successful validation, print a message to the console indicating success;
     - if there are no errors, return an empty string.
   - Implement exception handling to ensure the method is resilient to errors.
     - In a `catch (ValidationException e)` block, capture validation errors, print an error message to the error stream, and then return a formatted JSON string with information about the validation error.
     - In a `catch (IllegalAccessException e)` block, handle reflection-related errors and return a JSON message indicating an internal server error.

9. Create a package-less class `Main` with `public static void main(String[] args)` as the program entry point. In this method:

   - Initialize the processing component by creating an instance of `ObjectProcessor` and assigning it to the variable `processor`.
   - Prepare and run "Case 1: valid object" to verify the success path.
     - Print a case header to standard output.
     - Create `User validUser` using the constructor with parameters:
       * 1
       * "JanKowalski"
       * "jan.kowalski@example.com"
       * 30
       * "superSecret123"
     - Call `processor.process(validUser)` and leave the result printing within the logic of `process`;
     - finally, print a separator line between cases.
   - Prepare and run "Case 2: validation error (blank username)" to verify the reaction to a `@NotBlank` violation.
     - Print a case header to standard output.
     - Create `User invalidUserBlankName` with:
       * 2
       * " "

        * "anna.nowak@example.com"
        * 25
        * "password"
      - Call `processor.process(invalidUserBlankName)` and allow the error message to be generated by `process`;
      - end the section with a separator line.
    - Prepare and run "Case 3: validation error (age out of range)" to verify the reaction to a `@Range` violation.
      - Print a case header to standard output.
      - Create `User invalidUserAge` with:
        * 3
        * "PiotrZielinski"
        * "piotr.z@example.com"
        * 16
        * "anotherPass"
      - Call `processor.process(invalidUserAge)` and allow the error message to be generated by `process`;
      - end the section with a separator line.

10. Test that the application works correctly and present your results to the instructor.

11. In the `json` package, declare a public annotation type by creating the interface `JsonProperty`. Use the `@interface` construct to create a custom annotation.

    - Specify the annotation's lifetime so it is available during program execution. Set the retention to `RetentionPolicy.RUNTIME` so reflection tools can read the annotation.
    - Limit usage locations to class fields. Indicate `ElementType.FIELD` as the only allowed element type to which the annotation may be attached.
    - Add a required annotation element by defining the interface method `value` of type `String`. This element represents the field name in the JSON structure. Do not define a default value so the parameter is mandatory when using the annotation.

12. In the `json` package, declare a public annotation type by creating the interface `JsonIgnore`.

    - Specify a meta-annotation so that the annotation is available at runtime through reflection by using the enumeration constant `RetentionPolicy.RUNTIME`.
    - Specify a meta-annotation to limit usage exclusively to fields by using the enumeration constant `ElementType.FIELD`.

13. In the `json` package, declare a public annotation type by creating the interface `JsonSerializable` using the `@interface` construct and without additional elements.

    - Specify the annotation's lifetime with `RetentionPolicy.RUNTIME` so it is available at runtime and can be read via reflection mechanisms.

- Limit usage locations with `ElementType.TYPE` so the annotation can be added only to types (e.g., classes).

14. Introduce modifications to the `User` class in the `model` package:
    - Precede the class declaration with `@JsonSerializable`.
    - Precede the `id` variable declaration with `@JsonProperty` having the argument *userId*.
    - Precede the `password` variable declaration with `@JsonIgnore`.
15. In the `processing` package, declare a public class `JsonSerializer` responsible for converting objects annotated with JSON annotations into a textual format. The class is to use reflection for analyzing fields and interpreting metadata defined by annotations from the `json` package.
    - Implement a public static method `serialize(Object obj)` returning a `String`, which will carry out the serialization process. The method will delegate the `IllegalAccessException` to handle potential errors when accessing private fields.
      - In the initial part of the method, check the correctness of the provided argument.
        * Verify that the object `obj` is not `null`.
        * Check whether the object's class is annotated with `@JsonSerializable`.
        * If the annotation is missing, throw an `IllegalArgumentException` with a message indicating that serialization is not possible.
      - Create a `StringJoiner` to build the list of JSON elements separated by commas. Use the constructor `new StringJoiner(",")` to prepare the format for composing key–value pairs.
      - Iterate over all fields declared in the object's class using reflection.
        * Set accessibility for private fields with `field.setAccessible(true)`.
        * Skip fields annotated with `@JsonIgnore` so they are not included in serialization.
      - Determine the JSON key name for each field. If the field has the `@JsonProperty` annotation, obtain its value using `field.getAnnotation(JsonProperty.class).value()`. Otherwise, use the default field name returned by `field.getName()`.
      - Read the field value and format it depending on the type. If the value is an instance of `String`, enclose it in quotation marks. For other types, convert the value to text using `String.valueOf(value)`.
      - Add the formatted element to the `StringJoiner` as a key–value pair: `"̈keÿ:value"`. Each pair should be separated by a comma, in accordance with JSON object syntax.
      - After the iteration, return the complete JSON string by surrounding the joined elements with braces. Return the result as `"" + jsonElements.toString() + ""`.
16. Modify the code of the `ObjectProcessor` class in the `processing` package:

- After validation but before exception handling, start the process of serializing the object into JSON format using `JsonSerializer`.
  - Call `JsonSerializer.serialize(obj)` and assign its result to the variable `json`.
  - After completing serialization, print a message to the console indicating success;
  - then return the resulting JSON string.

17. Modify the code of the `Main` class by printing to the console the result obtained from calling the `process` method.

18. Test that the application works correctly and present your results to the instructor.

19. **Student Report (Observations and Conclusions).**

    - Prepare a separate report describing your observations and conclusions drawn from this assignment. Include:
      - a description of how annotation-based validation was planned and applied,
      - reflections on how reflection and annotations influenced the class design,
      - notes on the JSON serialization approach and field inclusion/exclusion decisions,
      - short comments on the three prepared cases in `Main`,
      - any proposals for further development or adjustments.